INTRODUCTION

In this report, we delve into the core concepts, architecture, and terminologies used in building large language models (LLMs).
LLMs are advanced AI systems designed to understand, generate, and work with human language in a highly sophisticated way. They're trained on vast amounts of text data—books, articles, websites, conversations—to learn patterns, structures, and context in language.

In this work we demonstrate the working and the execution of an AI chatbot and try to build a large language model which is used as the fundamental unit of this chatbot.

The chatbot is designed to read and interpret documents in PDF format — a challenging task that requires advanced understanding of natural language. To enable this, we make use of a language model that is built on a new and powerful method of processing text.

Earlier approaches to understanding language, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, were limited in their ability to handle long texts. These models processed text sequentially, word by word, which made them slow and sometimes inaccurate when dealing with longer or more complex sentences.

The use of attention-based language models makes it possible to build systems that read and understand documents more like humans. Our chatbot will be able to extract information, understand questions, and provide accurate answers by using a model trained to learn language patterns, context, and relevance.

This technology is especially useful in fields where reading and analyzing documents is a daily task. Whether it's legal documents, research papers, or official records, the chatbot can act as an intelligent assistant, helping users get the information they need quickly and accurately.

In machine learning, they are trained on massive amounts of text data and learn patterns, meanings, grammar, and even context behind words and sentences.

LLMs work using a deep learning architecture called the Transformer, which uses a mechanism called attention to focus on the most relevant parts of the text. This allows the model to understand not just individual words but also how they relate to each other across long passages.

In real-world applications, LLMs are used to build intelligent systems like chatbots, virtual assistants, search engines, and content analyzers — including the AI chatbot we are developing to read and understand PDF documents.

Background

Recurrent Neural Network:

Recurrent Neural Networks (RNNs) are a class of neural networks designed to handle **sequential data**, where the order of the elements matters—like time series, audio, or language.

Key Idea:

RNNs have **loops** that allow information to persist. This means they can take into account previous inputs when processing the current one—making them ideal for tasks like:

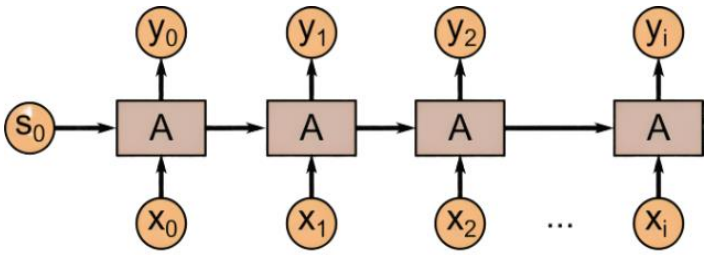
- **Language modeling**
- **Speech recognition**
- **Text generation**
- **Machine translation**

How They Work:

At each time step, an RNN takes:

- The current input Xo
- The **hidden state** from the previous step ht-1and updates the new hidden state ht, which captures both current and historical context.

However, RNNs struggle with **long-term dependencies** due to issues like **vanishing gradients**, which is why more advanced architectures like **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)** were developed. These versions have gates that control what information to keep or forget over time.



PROBLEMS WITH RNN:

1.Slow computation (for loop).

2.Vanishing or exploding gradients derivative (may be big or small).

3.Difficulty in accessing information from long time ago.

TRANSFORMER:

A transformer is a powerful type of deep learning model that changed how machines handle language, images, and even music. It was introduced in a 2017 paper titled

At its core, the transformer uses something called self-attention which helps the model figure out the importance of each word in a sentence by looking at all the words at once—rather than one by one. This ability to understand context deeply and in parallel makes it very effective for tasks like translation, text generation, and question answering.

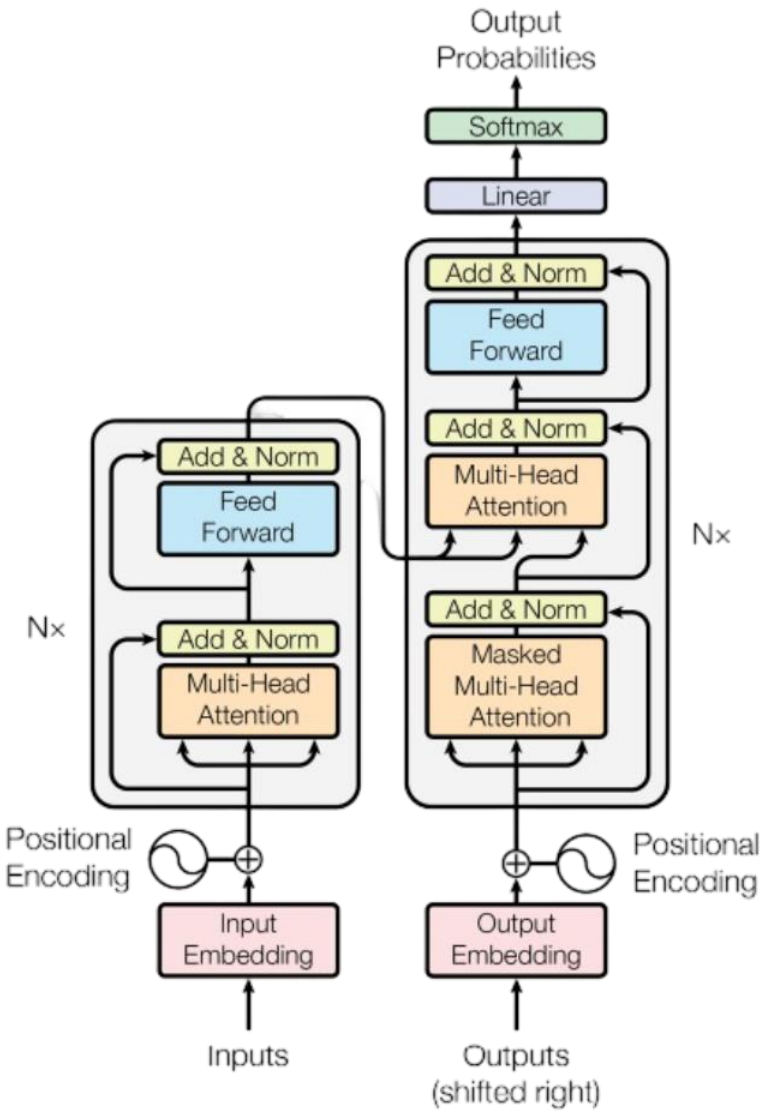


Figure 1: The Transformer - model architecture.

ENCODER:

In a transformer model, the **encoder** is one of the two main parts—the other being the decoder.

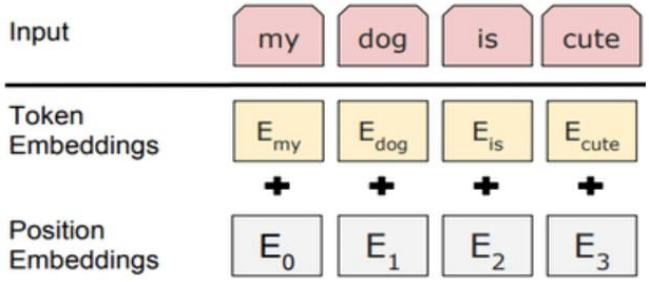
The encoder's job is to take input (like a sentence), understand its meaning, and convert it into a format the model can work with—a set of numerical representations called embeddings. Each encoder layer looks at all the words in the input at once using something called *self-attention*, which helps the model understand the relationships between words.

COMPONENTS :

1. INPUT EMBEDDING:

Input embedding is the process of converting words or tokens from the input text into numerical vectors that a transformer model can understand.

Imagine words like "cat," "car," or "courage"—they're just symbols to a computer. So, we turn each one into a list of numbers that captures its meaning based on how it's used in language. These vectors are called **embeddings**.



Why Do We Use INPUT Embeddings?

- Machines can't understand text directly — they understand numbers.
- Embeddings allow the model to capture semantic meaning (e.g., "king" is closer to "queen" than to "apple").
- These vectors help the model generalize and learn relationships between words.

How Does It Work?

1. Tokenization:
A sentence like “The cat sat” is broken into tokens: ["The", "cat", "sat"].

2. Embedding Lookup:
Each token is converted to a vector using an embedding matrix.
Example:
"The" → [0.1, 0.4, 0.3, ..., 0.7]

"cat" → [0.5, 0.6, 0.2, ..., 0.9]

"sat" → [0.3, 0.7, 0.1, ..., 0.4]

3. Embedding Matrix:
A large matrix (e.g., 30,000 tokens × 512 dimensions) that holds the vector representation of each token.
It is learned during training — the model figures out the best vectors to use to understand the language.

4. Adding Positional Encoding:
Since embeddings do not carry information about word order, positional encodings are added to the embeddings to help the model understand the sequence of the words.

5. Final Output:
The sum of word embedding and positional encoding is passed into the encoder's attention layer.

Mathematical View:

Let:

xi be the one hot encoder vector for token i.

E be the embedding matrix.

Then:

Embedding(xi)=xi.E

This return the **embedding vector** for the i-th token.

Real-World Example:

Sentence:
"Machine learning is powerful."

Tokenized:
["Machine", "learning", "is", "powerful", "."]

Embedding lookup gives:

"Machine" → [0.12, 0.55, ..., 0.34]

"learning" → [0.81, 0.33, ..., 0.25] …and so on

These vectors are passed into the encoder to start learning relationships and meanings.

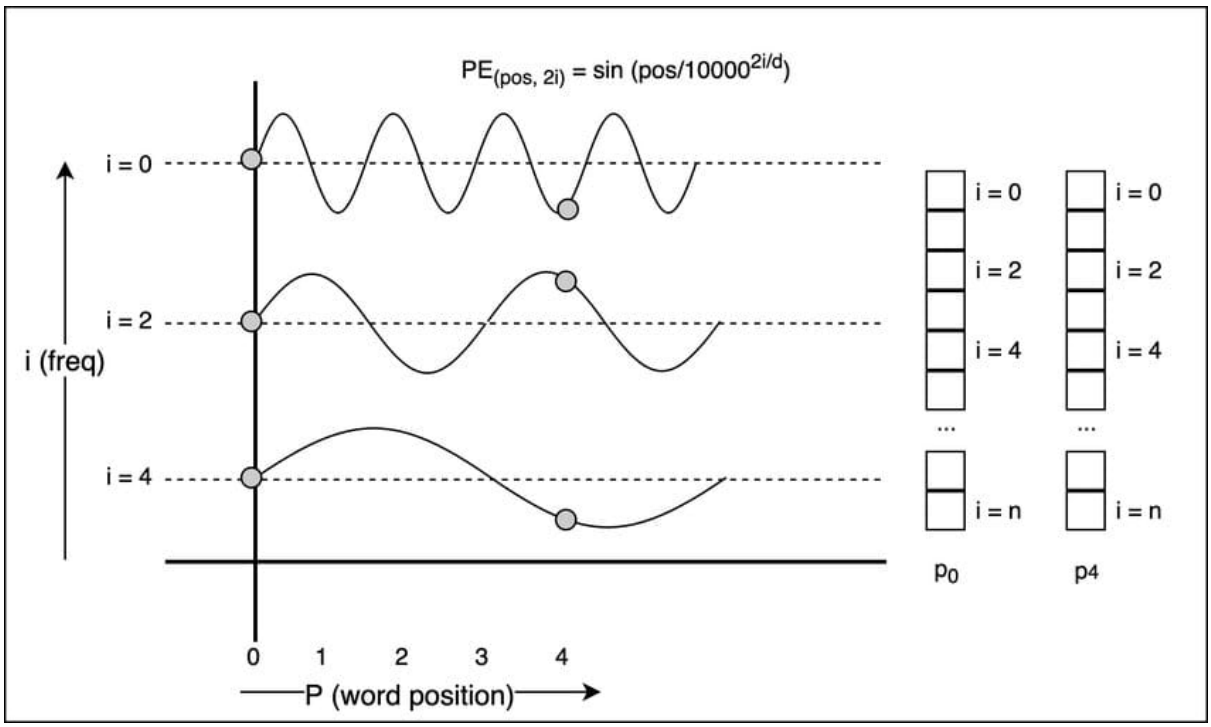
2. POSITIONAL ENCODING:

Positional encoding is a technique used in transformer models to help them understand the *order* of tokens (like words) in a sequence. Since transformers look at all words at once (not one by one), they don't inherently know which word comes first or last—which is critical for understanding meaning

FORMULA:

PE(pos,2i) = sin(pos/100002i/d)

PE(pos,2i+1) = cos(pos/100002i/d)



Why We Use Positional Encoding ?

Transformers, including the encoder, process all words in parallel (not one-by-one like RNNs or LSTMs). While this makes them faster, it also means they don't naturally understand the order of words.

- Problem: Transformers can't tell which word comes first, second, or last.
- Why It Matters: Word order affects meaning**.**

Solution: Add positional encoding to the word embeddings so the model knows the position of each word in the sequence.

How It Works ?

1. Tokenization:

Input sentence → split into tokens

E.g., "I love AI" → ["I", "love", "AI"]

2. Word Embedding:

Each token is converted into a vector that represents its meaning.

3. Generate Positional Encoding:

For each position (0, 1, 2, ...), a unique vector is generated using sine and cosine functions.

4. Add Embedding + Positional Encoding:

Final input to the encoder = word meaning + position info

This allows the encoder to process both what the word is and where it is.

Mathematical View:

Let:

: the word's position in the sentence (starting from 0)

: the dimension index

: total embedding size (e.g., 512)

The positional encoding formula is:

PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)

PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)

Real-Time Example :

Sentence:

“AI helps people.”

Word	Position	Word Embedding (2D for simplicity)	Positional Encoding	Final Vector
AI	0	[0.6, 0.2]	[0.00, 1.00]	[0.6, 1.2]
helps	1	[0.5, 0.4]	[0.84, 0.54]	[1.34, 0.94]
people	2	[0.7, 0.3]	[0.91, -0.42]	[1.61, -0.5]

3. SELF ATTENTION:

It’s a mechanism that allows the model to **focus on different parts of the input sentence when processing each word**. This helps the model grasp context—essential for tasks like translation, summarization, or answering questions.

In simple terms, self-attention allows each word to look at (or "attend to") every other word in the input to decide which words are important for understanding its own meaning.

FORMULA:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{dk})V$$

Where:

Q = Query matrix

K = Key matrix

V = Value matrix

Dk = dimension of the key vectors (used for scaling)

QKt = dot product between queries and keys

softmax = converts scores into probabilities

Why Do We Use Self-Attention?

In human language, words depend on each other to make sense. A single word can change meaning based on the words around it.

- Traditional Limitation:

Earlier models like RNNs and LSTMs processed sentences word-by-word in a sequence. This limited their ability to learn relationships between distant words, and they were slow due to sequential processing.

- The Transformer Solution:

Self-attention allows the model to

Process all words in parallel.

Understand relationships between all pairs of words.

Attend more to important words and less to irrelevant ones.

This improves both accuracy and efficiency in handling language tasks.

How Self-Attention Works ?

Let's understand how self-attention works using a sentence:

"The cat sat on the mat."

We want the model to understand how each word is related to the others.

Step 1: Input Embeddings

Each word is converted into a fixed-size vector (embedding).

For example:

"cat" → [0.5, 0.2, 0.7, …]

Step 2: Create Q, K, and V

For each word, we generate three vectors:

Query (Q): What this word wants to find.

Key (K): What this word offers to others.

Value (V): The actual content of the word.

These are generated using learned weight matrices applied to the embedding.

Step 3: Compute Attention Scores

For each word, compare its query to the keys of all words (including itself) using a dot product.

This gives a score that shows how relevant each other word is to the current word.

Step 4: Apply Softmax

The attention scores are passed through a softmax function to turn them into probabilities. These probabilities tell us how much attention the word should pay to others.

Step 5: Weighted Sum of Values

Each word's output is a weighted sum of all value vectors, weighted by the attention probabilities.

This final result is a new vector that represents the word in the context of the entire sentence.

REAL-TIME EXAMPLE:

- Let's look at a sentence:

"He dropped the glass because it was fragile."

The word "it" needs to refer to "glass". But how does the model know that?

- Self-attention lets the model:

Compare "it" to all previous words.

Find that "glass" has a high similarity with "it".

Assign a high attention weight between "it" and "glass".

Combine the "value" of "glass" into the output vector for "it".

This allows the model to understand that "it" = "glass" and not "he".

4. MULTI HEAD ATTENTION:

Multi-head attention is like giving the model multiple perspectives on the same sentence—each one focusing on different relationships between words. It's a key component of the transformer architecture.

Multi-head attention is a core component of the encoder in the Transformer architecture (used in models like GPT, BERT, etc.). It allows the model to attend to different parts of the input sequence simultaneously from different perspectives. This enhances the model's understanding of relationships within the data.

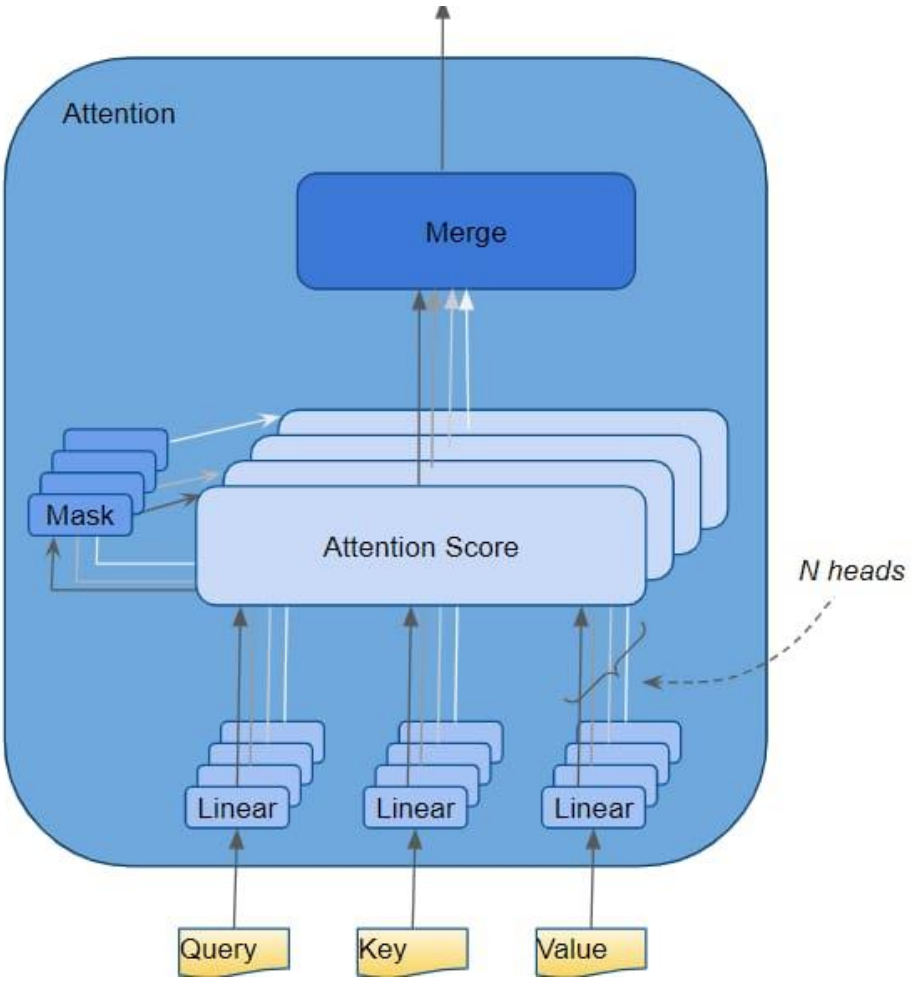
FORMULA:

MultiHead(Q,K,V) = Concat(head1,…,headn)WO

WHERE:

Headi = Attention(QWiQ,KWiK,VWiV)

- WiQ,WiK,WiVW^Q_i, W^K_i, W^V_i are learned projection matrices for each head
- WOW^O is the final projection matrix after concatenating the heads
- hh is the number of attention heads.



Why Do We Use Multi-Head Attention?

- Problem with Single Attention:

A single self-attention mechanism looks at only one aspect of word relationships — for example, just grammar or just topic similarity.

- Multi-Head Solution:

1. Look at different positions in the sentence simultaneously
2. Learn different kinds of patterns (like word order, syntactic structure, references)
3. Focus on multiple important words at the same time.

How Does Multi-Head Attention Work?

Let's say you have a sentence:

"The cat chased the mouse through the house."

Here's how multi-head attention processes it:

- Step 1: Input Embeddings

Each word is converted into a vector of size d_model (e.g., 512).

- Step 2: Linear Projections

The input embeddings are passed through different weight matrices to create separate Query (Q), Key (K), and Value (V) vectors for each head.

So for h heads, we create h sets of Q, K, and V:

Head 1: Q_1, K_1, V_1

Head 2: Q_2, K_2, V_2… up to Head h

Each head learns a different transformation of the input.

- Step 3: Scaled Dot-Product Attention

Each head performs self-attention separately:

head i = Attention (Q_i, K_i, V_i) = softmax(Q_iK_i^T/√d_k) V_i

- Step 4: Concatenation of Heads

All the heads' outputs are concatenated:

Concat(head_1, head_2, ..., head_h)

- Step 5: Final Linear Layer

This concatenated output is passed through a final linear layer (dense layer) to mix all the heads and produce the final output of the multi-head attention block.

Real-Time Example:

Sentence:

“The girl read the book because it was interesting.”

Let’s say:

- Head 1 focuses on grammatical structure: subject–verb–object
- Head 2 focuses on long-term dependencies: "it" → "book"
- Head 3 focuses on topic similarity: "read" ↔ "book"

Each head processes the same sentence but learns a different relationship.

5. ADDITION AND NORMALIZATION:

1. **Addition**: Add the input from the previous layer (residual connection) back to the output of the sublayer. This helps preserve the original information and stabilize learning.

   Output = LayerInput+SublayerOutput

2. **Layer Normalization**: Normalize the result to keep values in a consistent range—improving training speed and performance.

   FinalOutput = LayerNorm(Output)

IMPOTANCE:

- **Residual connections** help avoid vanishing gradients and allow deeper models to train more effectively.
- **Normalization** smooths out noisy activations and ensures stable behavior across tokens and batches.

How Addition and Normalization Works?

- What Happens in the Encoder?

In each encoder layer of the Transformer, there are two main sub-layers:

1. Multi-head self-attention

2. Feed-forward neural network (FFN)

After each of these sub-layers, the output is passed through a step called:

Addition and Normalization (also known as Residual Connection + LayerNorm)

This step ensures that:

The original input is preserved and added back to the output (Addition)

The combined output is normalized for stability (Normalization)

- Step-by-Step: How It Works

Let's say:

The input to a sub-layer is a vector

The sub-layer (attention or feed-forward) produces an output

Then the steps are:

- Step 1: Residual Addition

We add the input vector to the sub-layer's output:

$$z = x + \text{Sublayer}(x)$$

This is the residual connection. It ensures that the original input is not lost and helps with training deep networks.

- Step 2: Layer Normalization

Next, we normalize the result using LayerNorm:

Output = LayerNorm(z) = LayerNorm(x +Sublayer(x))

Layer normalization ensures that the output has:

A mean of 0

A standard deviation of 1

Then scaled and shifted using learned parameters (scale) and (bias)

How It Is Used in the Encoder:

In every layer of the Transformer encoder, there are two main operations:

1. Multi-Head Self-Attention

2. Feed-Forward Neural Network (FFN)

After each of these, the model applies Addition and Normalization, also known as:

- Residual Connection (Addition)
- Layer Normalization (Normalization)

This process helps the model preserve useful input information, maintain numerical stability, and train deeper layers more effectively.

Mathematical View:

Let:

: the input vector to a sub-layer

: output from a sub-layer (attention or FFN)

- Step 1: Residual Addition

  $z = x + \text{Sublayer}(x)$

- Step 2: Layer Normalization

Output = LayerNorm(z) = LayerNorm(x +Sublayer(x))

Real-Time Example:

Let’s say we are processing the sentence:

“AI transforms industries.”

- Step 1: Token Embeddings

"AI" → vector x1

"transforms" → x2

"industries" → x3

6. LAYER NORMALIZATION:

Layer normalization is a technique used in transformer models to stabilize and speed up training by normalizing the inputs across the features of each token’s vector—*not across the batch*, like batch normalization does.

FORMULA:

$$\text{LayerNorm}(x) = x-\mu/\sigma\cdot\gamma+\beta$$

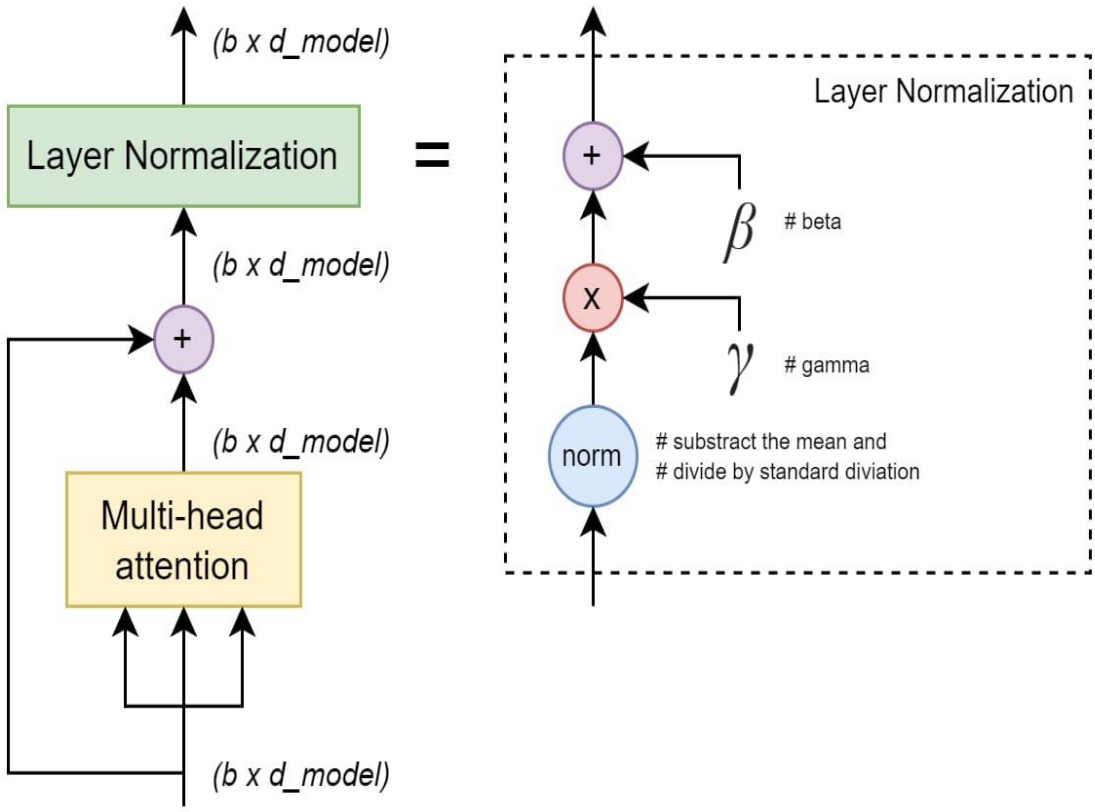
WHERE:

- μ\mu: Mean of the elements in the input vector
- σ\sigma: Standard deviation
- γ,β\gamma, \beta: Learnable scale and shift parameters (help the model adjust the normalized output)

WHERE IT USED:

Layer normalization appears **after each sub-layer** in a transformer block (like self-attention or feedforward), usually as part of the "Add & Norm" step:

1. Add the original input (residual connection).
2. Apply Layer Normalization to the Result.



How Does It Work?

Let's say you have a vector (e.g., the embedding of a word or token). LayerNorm works as follows:

1. Calculate the mean:

μ = 1/nΣn i=1 xi

2. Calculate the standard deviation:

σ =√1/nΣn i=1 (xi-μ)2

3. Normalize each feature:

ˆxi=xi-μ/σ

4. Scale and shift using learnable parameters:

Layer norm (xi)= γ. ˆxi+β

- B=shift(also learned)

The result is a vector where each component has zero mean and unit variance, scaled and shifted by trainable parameters.

7. FEED FORWORD:

The **feedforward layer** is a key building block inside each encoder and decoder block of a transformer. While attention mechanisms capture *relationships between words*, the feedforward network processes each position **independently** to deepen the model's understanding.

FEED FORWORD LAYER:

1. **Linear Transformation** The input embedding (or hidden state) is multiplied by a weight matrix:

   FFN(x)=max(0,xW1+b1)

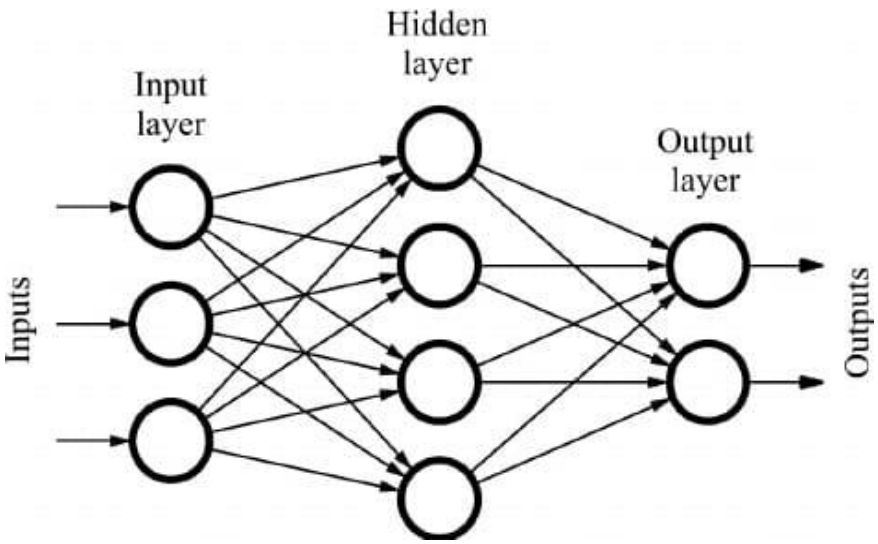
2. **Projection Back to Original Size** Then it's passed through another linear transformation:

   Output=(FFN(x))W2+b2

FORMULA:

FeedForward(x)=max(0,xW1+b1)W2+b2

- W1,W2W_1, W_2: learned weight matrices
- b1,b2b_1, b_2: learned biases
- max(0,·)\max(0, \cdot): ReLU activation



WORKING OF FEED FORWARD :

The Feed Forward Network (FFN) is applied after the attention layer in each transformer block

Step-by-step Working:

1. Input: Each token is represented as a vector (x) after self-attention and normalization.

2. First Linear Layer:

Multiplies x by weight matrix W_1 and adds bias b_1.

Formula: $z = xW_1 + b_1$

3. Activation Function:

Apply ReLU to add non-linearity: $a = ReLU(z)$

4. Second Linear Layer:

Transforms the activated output: $output = aW_2 + b_2$

5. Add & Normalize:

The result is added to the input (residual connection) and normalized.

USES OF FEED FORWARD:

1. Adds non-linearity to the model for better learning

2. Transforms attention outputs into more useful representations.

3. Processes each token independently to extract richer features.

4. Helps the model learn complex patterns and relationships.

5. Improves accuracy in tasks like translation, classification, and question answering.

6. Supports residual connections for stable and deep learning.

7. Enables feature scaling and dimensional adjustment (e.g., 512 → 2048 → 512).

8. Makes transformer layers more expressive and powerful.

REAL-TIME EXAMPLES:

Example 1: Machine Translation (English to French)

The FFN helps refine each translated word after attention has chosen the important context words.

Example 2: Chatbots

After attention figures out what part of your question is important, FFN helps reframe the response word-by-word before outputting.

Example 3: Sentiment Analysis

Attention focuses on emotion-bearing words.

FFN transforms them individually to classify as positive, negative, or neutral.

Example 4: Search Engines (BERT)

Each word in a query is processed by FFN after attention, making it better at understanding what users are searching for.

DATA PREPROCESSING:

Data preprocessing means preparing raw text so that it can be understood by a transformer model like the one which we are building . Transformer models do not understand human language directly — they only understand numbers (token IDs), so we have to convert the text into a format that the model can use.

IMPORTANCE OF DATA PREPROCESSING:

- Transformer encoder models work best when the input is:
- Clean and meaningful
- Broken into smaller word parts (tokens)
- Structured using special symbols like [CLS] and [SEP]
- Converted into numerical values
- Padded or trimmed to a fixed length

Execution Process :

- Extracting Text from PDF
- Text Cleaning
- Tokenization
- Feed into the encoder

1. Extracting Text from PDF:

This is the process of reading the text content from a PDF file. PDFs may contain text, images, or complex layouts. We use PDF processing tools (like pdfplumber, PyMuPDF, or PyPDF2) to read only the text part.

Example:

From a PDF page with:

- This is a sample PDF file.
- It contains text for processing.

The extracted text becomes a plain string:

- "This is a sample PDF file. It contains text for processing."

2. Text Cleaning:

Once we have the raw text, it often contains unwanted elements like punctuation, numbers, special characters, or inconsistent spacing. Text cleaning prepares it for analysis.

Cleaned Example:

"this is a sample pdf file it contains text for processing"

3. Tokenization:

Tokenization is the process of breaking down the text into smaller parts called tokens. These tokens can be:
Words (e.g., "this", "is", "sample")
Subwords (used in BERT, e.g., "un", "##believable")
The tokenizer converts each token into an ID number that the model understands.

Example Tokens:

['this', 'is', 'a', 'sample', 'pdf', 'file', 'it', 'contains', 'text', 'for', 'processing']

4. Feed into the encoder:

Feed into the encoder refers to the process of:

- Tokenizing the input text,
- Embedding each token,
- Adding positional information
- Passing the result through the encoder layers
- to obtain context-aware representations of the input.

Example:

Sentence: "I love AI"

- Tokenize → [101, 146, 1567, 7339, 102]
- Embed → Convert to vectors
- Add position info
- Pass through encoder layers
- Output: A vector for each word that knows its meaning in context.